

Grado en Ingeniería Informática
Tecnologías de Desarrollo de Software

Gestión de proyectos con Maven

Introducción

- Proceso de construcción de un **proyecto** implica:
 - Crear una **estructura de directorios**
 - Realizar tareas como compilar, ejecutar tests, **definir dependencias**, crear ejecutables, generar documentación, empaquetar y desplegar.
- Herramientas *build* (***build automation tools***) facilitan la construcción automatizando algunas tareas.
- Ejemplos de herramientas: make, ant, **maven**, **gradle**.
- Maven es la más utilizada en la actualidad, pero aumenta el uso de Gradle (elegida por Google para Android).
 - Según Snyk, Maven 78% y Gradle 38%

Extensible Markup Language (XML)

- Lenguaje de marcado (*markup language*).
- Datos **semi-estructurados**: esquema está implícito en los datos.
- Estructura **jerárquica**.
- Muy utilizado en la Web y para configuración de herramientas y sistemas.
- Útil para **serialización**: representar todo tipo de datos en un formato estándar.
- Representación de **metadatos**.

```
<?xml version="1.0" encoding="UTF-8"?>
<libro>
  <titulo> El Principito </titulo>
  <autor>
    <nombre>Antoine de Saint-Exupéry </nombre>
    <fecha_nacimiento> 29/06/1900 </fecha_nacimiento>
    <país>Francia</país>
  </autor>
  <fecha_publicacion> 1943 </fecha_publicacion>
  <editorial> Reynal & Hitchcock </editorial>
  <idioma> Francés </idioma>
  <paginas numero="93"/>
</libro>
```

Extensible Markup Language (XML)

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<libro>
```

```
  <titulo> El Principito </titulo>
```

```
  <autor>
```

```
    <nombre>Antoine de Saint-Exupéry </nombre>
```

```
    <fecha_nacimiento> 29/06/1900 </fecha_nacimiento>
```

```
    <país>Francia</país>
```

```
  </autor>
```

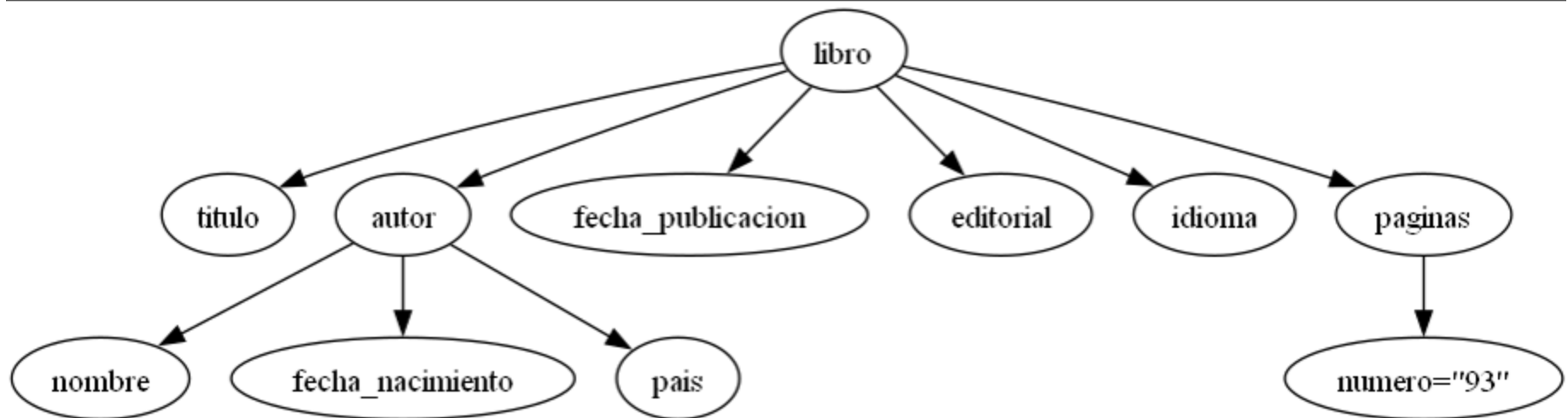
```
  <fecha_publicacion> 1943 </fecha_publicacion>
```

```
  <editorial> Reynal & Hitchcock </editorial>
```

```
  <idioma> Francés </idioma>
```

```
  <paginas numero="93"/>
```

```
</libro>
```



Maven: Características

Framework para automatizar la gestión de proyectos.

- Uso de **XML**
 - **Estructura predefinida** para los proyectos y flujos de tareas:
 - «*Convención sobre configuración*»
 - **Mecanismo de dependencias potente**
 - Dependencias son descargadas automáticamente de un repositorio local o del repositorio remoto Central Maven.
 - Basada en el concepto de “**ciclo de vida**”: conjunto de objetivos realizados ejecutando tareas implementadas por **plugins**.
 - **Extensible** vía plugins.
-
- Genera muchos **tipos de proyectos**: jar, war, ..
 - Genera **documentación** e informes sobre el proyecto, y el sitio web.
 - Javadoc, lista de dependencias, listas de mailing, informes de los tests, ..
 - Otros:
 - Integración simple de **JUnit** para ejecutar pruebas unitarias
 - Se puede usar desde **consola** o **IDEs** como Eclipse

Apache Maven Tutorial

 vogella.com/tutorials/ApacheMaven/article.html

Lars Vogel, Simon Scholz (c) 2013 - 2024 vogella GmbH – Version 2.5, 03.06.2021

This tutorial describes the usage and the core concepts of using Maven for building Java applications.

1. What is Apache Maven?

[Apache Maven](#) is an powerful build tool primary for Java software projects. It is implemented in Java which makes it platform-independent.

Using Maven within the Eclipse IDE - Tutorial

 vogella.com/tutorials/EclipseMaven/article.html

Lars Vogel (c) 2016 - 2024 vogella GmbH – Version 1.6, 01.08.2021

This tutorial describes the usage of Maven within the Eclipse IDE for building Java applications.

1. Using Maven with the Eclipse IDE

The Eclipse IDE provides support for the Maven build. This support is developed in the *M2Eclipse* project.



[Welcome](#)

[License](#)

ABOUT MAVEN

[What is Maven?](#)

[Features](#)

[Download](#)

[Use](#)



[Release Notes](#)

DOCUMENTATION

[Maven Plugins](#)

Documentation

Getting Started with Maven

- [Getting Started in 5 Minutes](#)
- [Getting Started in 30 Minutes](#)

Introductions

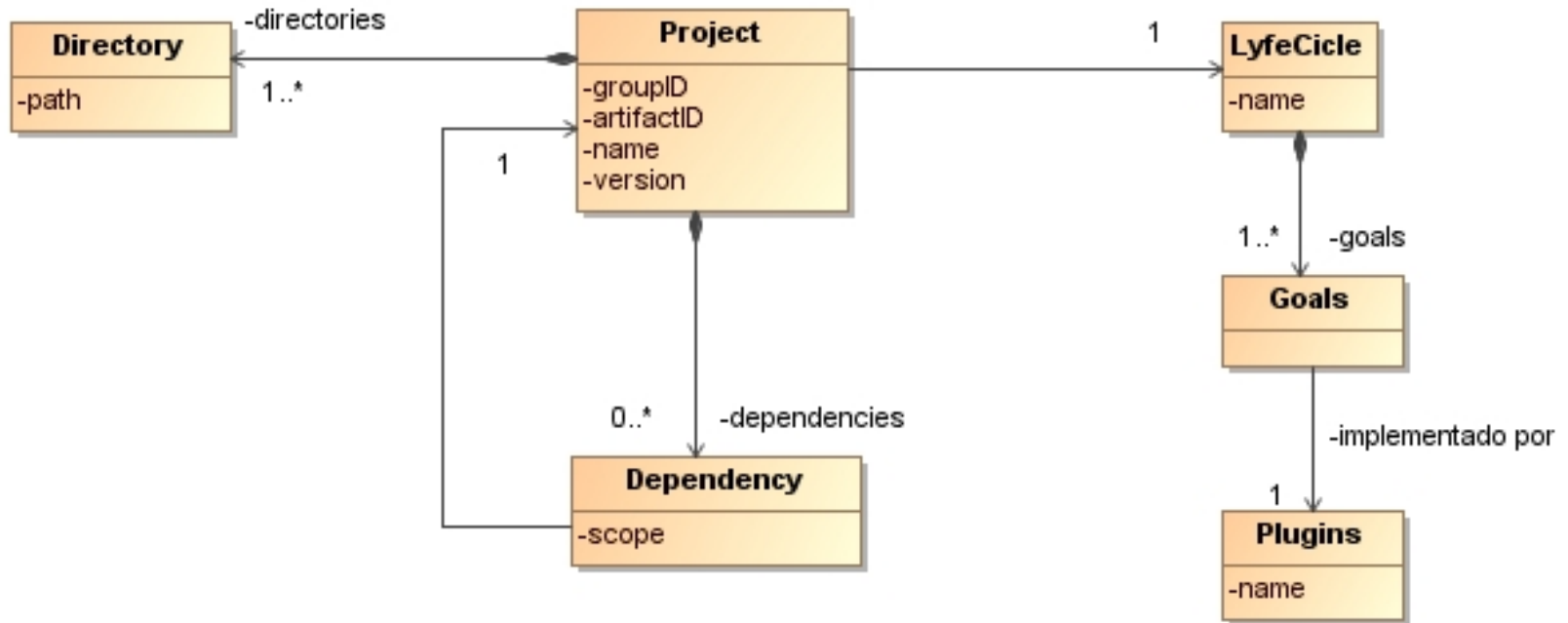
- [The Build Lifecycle](#)
- [The POM](#)
- [Profiles](#)
- [Repositories](#)
- [Standard Directory Layout](#)

POM: *Project Object Model*

- **Modelo conceptual de un proyecto**
 - **Fichero XML** (pom.xml en directorio raíz del proyecto) que contiene la información (metadatos) que configura al proyecto.
 - Nombre, directorios, dependencias, plugins, repositorios, informes, etc.
 - **Se genera automáticamente** y se puede modificar.
- Ejemplo de **POM** simple:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>umu.tds</groupId>
  <artifactId>MavenTestProject</artifactId>
  <name>Proyecto de prueba para Maven</name>
  <version>1.0</version>
  <packaging>jar</packaging>
</project>
```


Modelo Conceptual de maven



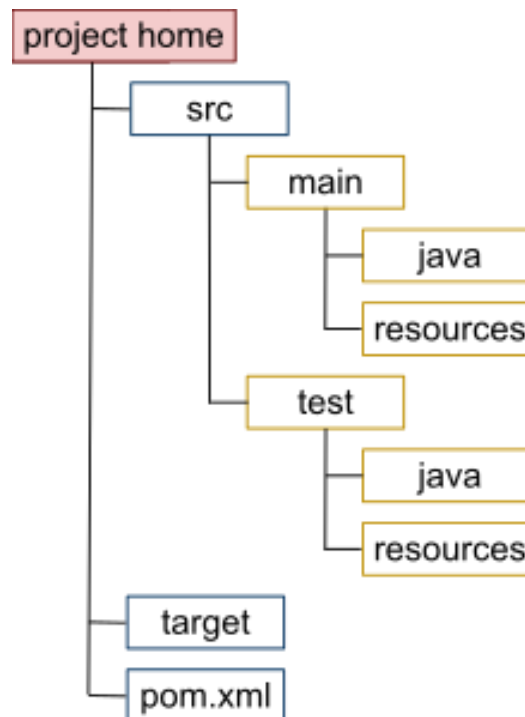
El archivo POM es una instancia del modelo conceptual de maven

POM para ejemplo de tutorial

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>tds.maven.apps</groupId>
  <artifactId>ProyectoMaven</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>ProyectoMaven</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Estructura de directorios

- Una estructura de directorios por defecto para todos los artefactos que componen un proyecto (puede ser modificada en el pom.xml).
- Directorio raíz con carpetas **src** (fuentes) y **target** (compilados y archivos generados: ejecutable, javadoc, informes, ..).



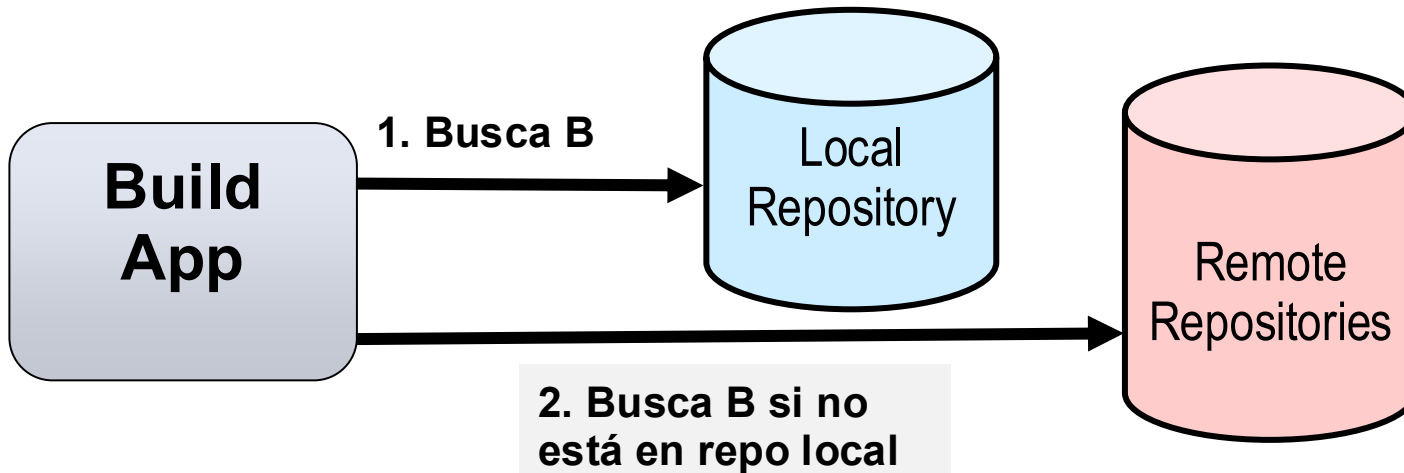
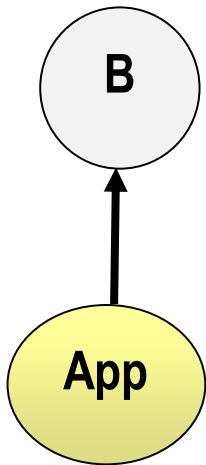
Dependencias

- Artefactos software (librerías, componentes, frameworks, etc.) requeridos en un proyecto, por ejemplo, el **Servicio de Persistencia**, **componente Swing para mostrar chat**, y **JCalendar** en AppChat, o **JUnit** comúnmente para las pruebas.
- Los artefactos no se añaden al proyecto, sino que se **referencian** de un repositorio central o local y Maven obtiene automáticamente las dependencias.
 - Manejo automático de la **transitividad** entre dependencias.
- Elementos que describen las dependencias:
 - groupId: identifica a la organización
 - artifactId: identifica al componente
 - version: versión actual
 - **scope**: ámbito en el que se usa (compile, test,..)
 - ...

Dependencias

« versión 1.0
o superior »

```
<dependencies>
  <dependency>
    <groupId>um.tds</groupId>
    <artifactId>B</artifactId>
    <version>[1.0,)</version>
    <scope>compile</scope>
  </dependency>
  ..
</dependencies>
```



Plugins



- Un ciclo de vida consta de un conjunto de goals implementados por **plugins** que se clasifican en categorías:
 - **Core**: clean, compiler, deploy, install, surefire, site,...
 - **Report**: javadoc, pmd, checkstyle,...
 - **Packaging types/tools**: ear, jar, war, rar, source,...
 - **Tools**: artifact, archetype, dependency, plugin,...
- **Ejecutar** un goal de un plugin **en consola**:
 - **mvn [nombre plugin]:[nombre goal]**
 - Ejemplos: **mvn compile**, **mvn install**
- Forman el núcleo de Maven: “todo el trabajo se consigue mediante ejecución de plugins”.
- Se pueden instalar automáticamente o se declaran en el POM.
- Cuando se ejecuta un comando mvn, Maven descarga el plugin desde el repositorio central si no está ya descargado en el local.

Welcome

License

ABOUT MAVEN

What is Maven?

Features

Download

Use

Release Notes

DOCUMENTATION

Maven Plugins

Maven Extensions

Index (category)

User Centre

Plugin Developer Centre

Maven Repository Centre

Maven Developer Centre

Books and Resources

Security

COMMUNITY

Community Overview

Project Roles

How to Contribute

Getting Help

Issue Management

Getting Maven Source

The Maven Team

PROJECT DOCUMENTATION

Project Information

MAVEN PROJECTS

Archetype

Artifact Resolver

Doxia

Extensions

JXR

Maven

Parent POMs

Plugins

Plugin Testing

Plugin Tools

Available Plugins

Maven is - at its heart - a plugin execution framework; all work is done by plugins. Looking for a specific goal to execute? This page lists the core plugins and others. There are the build and the reporting plugins:

- Build plugins** will be executed during the build and they should be configured in the `<build/>` element from the POM.
- Reporting plugins** will be executed during the site generation and they should be configured in the `<reporting/>` element from the POM. Because the result of a Reporting plugin is part of the generated site, Reporting plugins should be both internationalized and localized. You can read more about the [localization of our plugins](#) and how you can help.

Supported By The Maven Project

To see the most up-to-date list browse the Maven repository, specifically the `org/apache/maven/plugins` subfolder. *(Plugins are organized according to a directory structure that resembles the standard Java package naming convention)*

Plugin	Type*	Version	Release Date	Description	Source Repository	Issue Tracking
Core plugins				Plugins corresponding to default core phases (ie. clean, compile). They may have multiple goals as well.		
<code>clean</code>	B	3.2.0	2022-04-01	Clean up after the build.	Git / GitHub	Jira MCLEAN
<code>compiler</code>	B	3.10.1	2022-03-11	Compiles Java sources.	Git / GitHub	Jira MCOMPILER
<code>deploy</code>	B	3.0.0	2022-07-16	Deploy the built artifact to the remote repository.	Git / GitHub	Jira MDEPLOY
<code>failsafe</code>	B	3.0.0-M7	2022-06-07	Run the JUnit integration tests in an isolated classloader.	Git / GitHub	Jira SUREFIRE
<code>install</code>	B	3.0.1	2022-07-17	Install the built artifact into the local repository.	Git / GitHub	Jira MINSTALL
<code>resources</code>	B	3.3.0	2022-07-23	Copy the resources to the output directory for including in the JAR.	Git / GitHub	Jira MRESOURCES
<code>site</code>	B	4.0.0-M3	2022-07-25	Generate a site for the current project.	Git / GitHub	Jira MSITE
<code>surefire</code>	B	3.0.0-M7	2022-06-07	Run the JUnit unit tests in an isolated classloader.	Git / GitHub	Jira SUREFIRE
<code>verifier</code>	B	1.1	2015-04-14	Useful for integration tests - verifies the existence of certain conditions.	Git / GitHub	Jira MVERIFIER
Packaging types/tools				These plugins relate to packaging respective artifact types.		
<code>ear</code>	B	3.3.0	2022-10-18	Generate an EAR from the current project.	Git / GitHub	Jira MEAR
<code>ejb</code>	B	3.2.1	2022-04-18	Build an EJB (and optional client) from the current project.	Git / GitHub	Jira MEJB
<code>jar</code>	B	3.3.0	2022-09-12	Build a JAR from the current project.	Git / GitHub	Jira MJAR
<code>rar</code>	B	3.0.0	2022-07-17	Build a RAR from the current project.	Git / GitHub	Jira MRAR
<code>war</code>	B	3.3.2	2021-09-10	Build a WAR from the current project.	Git / GitHub	Jira MWAR
<code>app-client/acr</code>	B	3.1.0	2018-06-19	Build a JavaEE application client from the current project.	Git / GitHub	Jira MACR
<code>shade</code>	B	3.4.1	2022-10-27	Build an Uber-JAR from the current project, including dependencies.	Git / GitHub	Jira MSHADE
<code>source</code>	B	3.2.1	2019-12-21	Build a source-JAR from the current project.	Git / GitHub	Jira MSOURCES

Estructura del Proyecto (Archetypes)

- Define el **tipo de proyecto** que se quiere crear.
 - Determina una estructura de proyecto y POM
 - Ficheros XML que son la entrada a *templates* de generación de código.
- Ejemplos:
 - maven-archetype-quickstart** (básico y usado para AppChat)
 - maven-archetype-j2ee-simple
 - maven-archetype-webapp
- Orden en **consola**:
 - mvn archetype:generate** -DgroupId=umu.tds
 - DartifactId=proyectoJ2eeTDS
 - DarchetypeArtifactId=maven-archetype-quickstart

Ciclos de vida. Fases de Build

- Basado en el concepto de **ciclo de vida**: *build, clean y site*.
 - Cada ciclo de vida consta de varias fases.

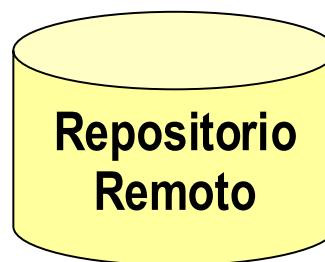
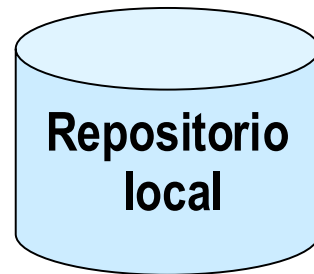
Fases de Build

- **validate**: valida que el proyecto esté correcto y tiene toda la información necesaria para su construcción.
- **compile**: compila el código fuente del proyecto.
- **test**: lanza las pruebas unitarias
- **package**: toma las clases compiladas y recursos y crea un paquete con el proyecto (jar, war, ear)
- **integration-test**: ejecuta otros tests que requieren el paquete
- **verify**: realiza algún tipo de chequeo para comprobar si el paquete cumple unas normas de calidad.
- **install**: instala el paquete en el repositorio local para ser usado como dependencia por otros proyectos localmente.
- **deploy** : copia el paquete a un repositorio remoto para ser compartido con otros usuarios y proyectos.

Repositorios de artefactos

- Almacenan todo tipo de artefactos (incluido nuestro proyecto) en una estructura jerárquica (organizada por groupId o paquete): JAR, EAR, WAR, EJBs, ZIPs, plugins, etc.
 - Ejemplo: `<usuario-win>/ .m2/repository/com/toedter/jcalendar/1.4/`
- Dependencias se buscan primero en el repositorio local (por lo general `.m2/repository` en el directorio de usuario). Si no se encuentran allí, se buscan en el repositorio remoto (Maven central por defecto, no debe ser indicado en POM, salvo si se cambia URL)

`<usuario-win>/
.m2/repository/`



 search.maven.org

<https://central.sonatype.com/?smo=true>

<https://mvnrepository.com/>

Find OSS Components

As stewards of Central for nearly 20 years and inventors of both software supply chain management and Nexus Repository, Sonatype knows that the integrity of your build is critical.

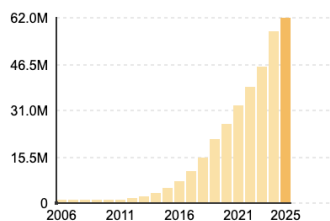
[Advanced Options](#)

Get recommended versions right

Avoid getting bogged down with re-work

Optimize your workflow with prioritized fixing, by spotlighting actionable and relevant findings only. No noisy alerts, just easy dependency upgrade suggestions.

Indexed Artifacts (61.9M)



Popular Categories

Testing Frameworks & Tools

Android Packages

JVM Languages

Logging Frameworks

Java Specifications

JSON Libraries

Core Utilities

Mocking

What's New in Maven

**ZIO AWS Core**[dev.zio](#) » [zio-aws-core](#) » 7.33.11.1

Low-level AWS wrapper for ZIO

Last Release on Sep 18, 2025

438 usages

[APL](#) [Apache](#)**Jeap Archrepo Importers**[ch.admin.bit.jeap](#) » [jeap-archrepo-importers](#) » 2.5.3

Jeap Archrepo Importers

Last Release on Sep 18, 2025

11 usages

[Apache](#)**ZIO AWS DynamoDB**[dev.zio](#) » [zio-aws-dynamodb](#) » 7.33.10.1

Low-level AWS wrapper for ZIO

Last Release on Sep 18, 2025

9 usages

[APL](#) [Apache](#)



C:\Users\jesus\.m2\repository\com\toedter\jcalendar\1.4

Nombre	Fecha de modificación	Tipo	Tamaño
 _remote.repositories	01/09/2022 2:41	Archivo REPOSITORIES	1 KB
 jcalendar-1.4.jar	01/09/2022 2:41	Executable Jar File	162 KB
 jcalendar-1.4.jar.sha1	01/09/2022 2:41	Archivo SHA1	1 KB
 jcalendar-1.4.pom	01/09/2022 2:41	Archivo POM	2 KB
 jcalendar-1.4.pom.sha1	01/09/2022 2:41	Archivo SHA1	1 KB
 jcalendar-1.4-sources.jar	01/09/2022 2:41	Executable Jar File	102 KB
 jcalendar-1.4-sources.jar.sha1	01/09/2022 2:41	Archivo SHA1	1 KB
 m2e-lastUpdated.properties	01/09/2022 2:41	Archivo PROPERTIES	1 KB

Using Maven within the Eclipse IDE - Tutorial

 Lars Vogel (c) 2016 - 2022 vogella GmbH – Version 1.6, 01.08.2021

This tutorial describes the usage of Maven within the Eclipse IDE for building Java applications.

1. Using Maven with the Eclipse IDE

The Eclipse IDE provides support for the Maven build. This support is developed in the *M2Eclipse* project.

It provides an editor for modifying the pom file and downloads dependencies if required. It also manages the classpath of the projects in the IDE. You can also use wizards to import existing Maven projects and to create new Maven projects.

2. Installation and configuration of Maven for Eclipse

2.1. Installation of the Maven tooling for the Eclipse IDE

Most Eclipse IDE downloads already include support for the Maven build system. To check, use **Help > About** and check if you can see the Maven logo (with the **M2E** sign).



If Maven support is not yet installed, the following description can be used to install it.

► [Installation of Maven support into the Eclipse IDE](#)

LoginRegistro

src

umu.tds

umu.tds.controlador

umu.tds.dao

umu.tds.dominio

RepositorioUsuario

Usuario.java

umu.tds.gui

JRE System Library [J2SE]

Referenced Libraries

Maven Dependencies

bin

lib

target

pom.xml

Dependencies

Dependencies

driverPersistencia : 2.0



Add...

Remove

Properties...

Manage...

To manage your transitive dependency exclusions, please use the [Dependency Hierarchy](#) page.

Overview

Dependencies

Dependency Hierarchy

Effective POM

pom.xml

```
1<project xmlns="http://maven.apache.org/POM/4.0.0"
2  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/
4
5  <modelVersion>4.0.0</modelVersion>
6  <groupId>umu.tds</groupId>
7  <artifactId>LoginRegistro</artifactId>
8  <version>0.0.1-SNAPSHOT</version>
9
10 <dependencies>
11   <dependency>
12     <groupId>umu.tds</groupId>
13     <artifactId>driverPersistencia</artifactId>
14     <version>2.0</version>
15   </dependency>
16 </dependencies>
17 </project>
```

Package Explorer ×

- instagram
- LoginRegistro
 - src
 - umu.tds
 - umu.tds.controlador
 - umu.tds.dao
 - umu.tds.dominio
 - RepositorioUsuarios.java
 - Usuario.java
 - umu.tds.gui

1 VentanaPrincipal

2 Login

3 Calculator

4 EjemploBotonesGBL

5 GridBagLayoutDemo

Run As

Run Configurations...

Organize Favorites...

Prin... PanelItemFot... PanelNuevaFo...

encies

encies

1 Maven build Alt+Shift+X, M

2 Maven build...

3 Maven clean

4 Maven generate-sources

5 Maven install

6 Maven test

ANEXO

Maven desde la consola

Maven desde consola

1. Descargar maven (fichero Binary.zip) y descomprimir en C:\Program Files
2. Se puede crear un directorio para repositorio local o **mantener el repositorio por defecto**. En Windows: usuarios/<usuario>/ .m2/repository
3. Crear y ejecutar un archivo .bat como el de abajo (se debería cambiar la version 3.6.3 de maven a la actual):

```
SET M2_HOME=C:\Program Files\apache-maven-3.6.3
SET M2=C:\Program Files\apache-maven-3.6.3\bin
SET PATH=%M2%;%JAVA_HOME%\bin;%PATH%
SET JAVA_HOME=C:\Program Files\Java\“Directorio con Java JDK”
```

4. Ejecutar `mvn -version`, debería aparecer un texto similar al de abajo:

```
Apache Maven 3.6.3 (40f52333136460af0dc0d7232c0dc0bcf0d9e117; 2019-08-27T17:06:16+02:00)
Maven home: C:\Program Files\apache-maven-3.6.3\bin\..
Java version: 1.8.0_172, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk1.8.0_172\jre
Default locale: es_ES, platform encoding: Cp1252
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
```

Maven desde consola

5. Crear un proyecto Maven, ejecutando el siguiente comando:

```
mvn archetype:generate -DgroupId=umu.tds.apps  
-DartifactId=AppGrupo3  
-DarchetypeArtifactId=maven-archetype-quickstart
```

Responder pulsando tecla “Enter” cada vez que se detenga la ejecución para pedir una entrada al usuario. **Se instalarán varios plugins** en el repositorio y se creará el directorio \AppGrupo3 en el directorio actual con la carpeta src y el archivo pom.xml

6. Ejecutar **mvn compile** en el directorio \AppGrupo3 que contiene el archivo pom.xml. Se crea la carpeta target con el código compilado.

7. Se pueden ir ejecutando comandos Maven: test, install, package,...